

## Sprint Documentation

|                              |                                                    |                             |
|------------------------------|----------------------------------------------------|-----------------------------|
| <b>Sprint 17</b>             | <b>Start Date: 12/05/26</b>                        | <b>Final Date: 14/05/26</b> |
| <b>Project Name:</b>         | <b>civitas-backend</b>                             |                             |
| <b>Ano: 5º</b>               | <b>Análise e Desenvolvimento de Sistemas - AMS</b> |                             |
| <b>Team Members</b>          |                                                    |                             |
| João Antonio Gomes Cestonaro |                                                    |                             |
| João Pedro Chapichi          |                                                    |                             |
|                              |                                                    |                             |
|                              |                                                    |                             |

## Sprint Backlog

| Task # | Description            | Start Date | Final date |
|--------|------------------------|------------|------------|
| 100    | Correções em Documento | 12/05/26   | 14/05/26   |
|        |                        |            |            |
|        |                        |            |            |
|        |                        |            |            |

## Task Description

| Task # | Description                                                                                                                                                                                                                                                                                                      | Assigned To                                       | Status | Estimated Hours | Logged Hours |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|--------|-----------------|--------------|
| 100    | Refatorar completamente o fluxo de armazenamento de documentos do sistema, removendo a estrutura isolada de Documento e incorporando o armazenamento diretamente na entidade Despesa, com upload seguro de arquivos, geração de hash baseada no conteúdo do documento e controle de duplicidade com confirmação. | João Antonio Gomes Cestonaro, João Pedro Chapichi | Done   | 8               | 8            |

|  |  |  |  |  |  |
|--|--|--|--|--|--|
|  |  |  |  |  |  |
|  |  |  |  |  |  |

## Sprint Description

### Tarefa: Sprint 17 - Correções em Documento

#### Objetivo

Refatorar completamente o fluxo de armazenamento de documentos do sistema, removendo a estrutura isolada de Documento e incorporando o armazenamento diretamente na entidade Despesa, com upload seguro de arquivos, geração de hash baseada no conteúdo do documento e controle de duplicidade com confirmação.

#### Descrição

Essa task tem como objetivo alterar a arquitetura atual de armazenamento de documentos do sistema.

Atualmente existe uma estrutura própria para Documento, contendo model, DTOs, builders, services e repositories específicos.

Essa estrutura deverá ser removida completamente, pois não existirá mais uma tabela dedicada para documentos.

A partir dessa alteração, os dados do documento passarão a ser armazenados diretamente na entidade Despesa.

O desenvolvedor deverá ajustar toda a estrutura relacionada à Despesa para suportar o novo fluxo de upload e armazenamento de documentos.

A entidade Despesa deverá receber dois novos campos:

- NomeDocumento
- HashDocumento

O sistema deverá permitir upload de arquivos.

O hash do documento deverá ser gerado exclusivamente pelo backend, utilizando o conteúdo real do arquivo recebido.

Esse hash será utilizado como identificador único do documento e também como nome físico do arquivo salvo internamente no sistema.

Caso o sistema identifique que já existe outra despesa utilizando o mesmo documento (mesmo hash), o backend não deverá bloquear automaticamente o cadastro.

Nesse cenário, o backend deverá encerrar a requisição e retornar uma resposta indicando que o documento já existe e que é necessária confirmação antes da conclusão do cadastro.

Caso a confirmação seja enviada em uma nova requisição, a despesa deverá ser salva normalmente utilizando o mesmo documento já existente.

Se o arquivo físico já existir no sistema, ele não deverá ser salvo novamente.

#### Critérios de Aceite

- Remoção da estrutura Documento
- Remover completamente a estrutura dedicada de Documento do sistema
- Remover ModelDocumento
- Remover DTOs relacionados a Documento
- Remover Builders relacionados a Documento
- Remover Services relacionados a Documento
- Remover Repositories relacionados a Documento
- Remover qualquer referência, dependência ou relacionamento relacionado à antiga estrutura de Documento
- Garantir que não existam dependências quebradas após a remoção

### **Alterações na entidade Despesa**

- Adicionar os novos campos na entidade Despesa:
  - NomeDocumento
  - HashDocumento
- Os campos deverão permitir valor nulo caso a despesa não possua documento anexado
- Ajustar migration correspondente
- Garantir persistência correta dos novos campos no banco de dados

### **Alterações nos DTOs de Despesa**

- Ajustar DTOs de criação, edição e retorno da entidade Despesa
- Permitir recebimento de documento via upload
- O DTO deverá receber o arquivo utilizando tipo compatível com upload (IFormFile)
- Adicionar campo:
  - ConfirmarDocumentoDuplicado
- O campo ConfirmarDocumentoDuplicado deverá possuir valor padrão false
- O hash não deverá ser enviado manualmente
- O hash deverá ser gerado exclusivamente pelo backend
- Garantir retorno dos campos NomeDocumento e HashDocumento quando necessário

### **Alterações nos Builders de Despesa**

- Ajustar builders relacionados a Despesa para trabalhar com os novos campos
- Garantir mapeamento correto entre DTOs e entidade
- Garantir que NomeDocumento e HashDocumento sejam preenchidos apenas após o processamento do arquivo pelo backend

### **Alterações nos Controllers de Despesa**

- Ajustar endpoint de criação/edição para receber upload de arquivo
- Ajustar binding do endpoint para leitura correta de IFormFile

### **Alterações nos Services de Despesa**

- Ajustar services da entidade Despesa para suportar upload de documentos
- Implementar geração automática do hash com base no conteúdo real do arquivo
- Não utilizar nome do arquivo como base para geração do hash

- Persistir NomeDocumento e HashDocumento na despesa
- Renomear o arquivo utilizando o hash gerado
- Salvar o arquivo fisicamente apenas caso ele ainda não exista
- Caso o arquivo já exista fisicamente, reutilizar o arquivo existente

## Fluxo esperado

### Cadastro sem duplicidade

- Receber dados da despesa e arquivo enviado
- Ler conteúdo do arquivo recebido
- Gerar hash baseado no conteúdo do documento
- Verificar se já existe despesa com o mesmo HashDocumento
- Caso não exista:
  - salvar o arquivo fisicamente
  - salvar a despesa normalmente
  - persistir NomeDocumento e HashDocumento

### Cadastro com duplicidade

- Caso exista outra despesa com o mesmo HashDocumento, verificar o campo ConfirmarDocumentoDuplicado
- Se ConfirmarDocumentoDuplicado = false:
  - não salvar a despesa
  - não salvar novo arquivo
  - encerrar a requisição
  - retornar resposta solicitando confirmação
- Caso uma nova requisição seja enviada com ConfirmarDocumentoDuplicado = true:
  - permitir salvamento da despesa
  - reutilizar o mesmo HashDocumento
  - não salvar nova cópia física do arquivo

## Regra de confirmação de documento duplicado

- O sistema deve permitir reutilização do mesmo documento em despesas diferentes
- O sistema não deve bloquear permanentemente o uso de documento já existente
- A validação de duplicidade deve ocorrer utilizando HashDocumento
- O backend não deve permanecer aguardando interação externa
- O backend deve responder e encerrar normalmente
- A confirmação deverá ocorrer por meio de nova requisição contendo a sinalização apropriada
- Exemplo de resposta esperada

```
{
  "sucesso": false,
  "requerConfirmacao": true,
  "codigo": "DOCUMENTO_DUPLICADO",
  "mensagem": "Este documento já está vinculado a outra despesa. Deseja continuar?"
}
```

## Alterações nos Repositories de Despesa

- Ajustar repository caso necessário para persistência dos novos campos
- Criar ou ajustar método para busca por HashDocumento
- Permitir verificação de duplicidade antes do salvamento
- Garantir compatibilidade com a nova estrutura

## Armazenamento físico dos arquivos

- Criar pasta interna no sistema para armazenamento dos documentos
- Garantir que os arquivos sejam armazenados apenas nessa pasta
- O nome físico do arquivo deverá ser o hash gerado
- O nome original não deverá ser utilizado como nome físico
- O nome original deverá ser armazenado apenas em NomeDocumento
- Caso o arquivo já exista fisicamente, ele não deverá ser salvo novamente
- Caso não exista, ele deverá ser salvo normalmente

## Observações e Sugestões

- O hash deverá ser gerado exclusivamente pelo backend
- Recomenda-se utilizar SHA-256 para geração do hash
- O hash deverá ser calculado com base no conteúdo real do arquivo
- Validar extensão, tamanho máximo e segurança dos arquivos enviados
- Garantir tratamento adequado para atualização ou substituição de documentos em despesas já existentes
- Garantir que a lógica de reutilização do documento não gere duplicidade física desnecessária

## Sprint Results

- Inicialmente foi removido todas as instâncias e referências da classe Documento dentro do programa, adicionalmente removendo conexão desnecessária com o Fornecedor.
- Depois foi alterado a model de Despesa para ter os dois campos novos referentes a NomeDocumento e HashDocumento, junto com a alteração de tipo do campo ValoresAdicionais para String?.

```

using Civitas.WebAPI.Objects.Enums;
using System.ComponentModel.DataAnnotations.Schema;

namespace Civitas.WebAPI.Objects.Models
{
    /// <summary>
    /// Entidade central que representa uma obrigação financeira, conta ou fatura a ser paga.
    /// Mapeia a tabela 'despesa' do banco de dados.
    /// </summary>
    [Table("despesa")]
    44 referências
    public class Despesa
    {
        /// <summary>
        /// Identificador único da despesa (Chave Primária).
        /// </summary>
        [Column("id")]
        6 referências
        public int Id { get; set; }

        /// <summary>
        /// Número de identificação impresso no documento fiscal (NF, Fatura, Boleto).
        /// </summary>
        [Column("numerodocumento")]
        4 referências
        public string NumeroDocumento { get; set; }

        /// <summary>
        /// Nome de identificação do documento
        /// </summary>
        [Column("nomedocumento")]
        3 referências
        public string? NomeDocumento { get; set; }

        /// </summary>
        [Column("valoresopcionais")]
        2 referências
        public string? ValoresOpcionais { get; set; }
    }
}
0 referências

```

- Alterado também a DTO de Despesa para que os 2 campos fossem adicionados, além de dois campos para importação de documento (IFormFile Documento) e ConfirmarDocumentoDuplicado do tipo Boolean.

```

/// <summary>
/// Nome do documento fornecido pelo usuário
/// </summary>
2 referências
public string? NomeDocumento { get; set; }

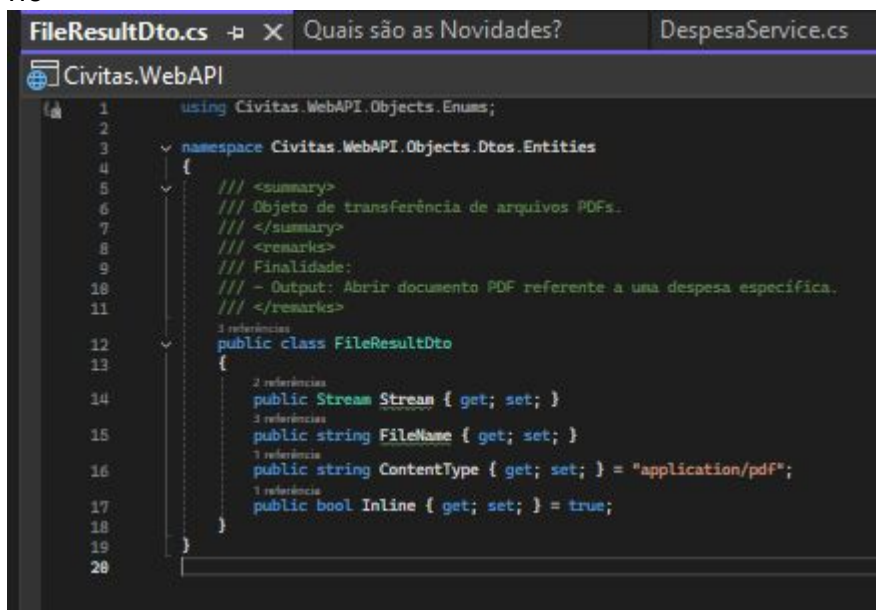
/// <summary>
/// Hash SHA-256 do documento (gerado automaticamente pelo código)
/// </summary>
5 referências
public string? HashDocumento { get; set; }

/// <summary>
/// Campo para confirmação de documento duplicado.
/// </summary>
1 referência
public bool? ConfirmarDocumentoDuplicado { get; set; } = false;

/// <summary>
/// Campo de arquivo de Documento para upload.
/// </summary>
7 referências
public IFormFile? Documento { get; set; }

```

- Adicionado uma classe nova para transferência de arquivos PDFs para visualização no navegador.

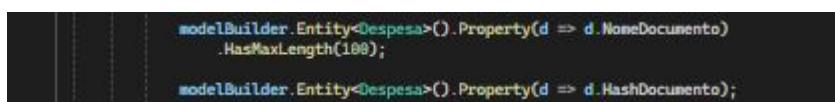


```

1 using Civitas.WebAPI.Objects.Enums;
2
3 namespace Civitas.WebAPI.Objects.Dtos.Entities
4 {
5     /// <summary>
6     /// Objeto de transferência de arquivos PDFs.
7     /// </summary>
8     /// <remarks>
9     /// Finalidade:
10    /// - Output: Abrir documento PDF referente a uma despesa específica.
11    /// </remarks>
12    public class FileResultDto
13    {
14        public Stream Stream { get; set; }
15        public string FileName { get; set; }
16        public string ContentType { get; set; } = "application/pdf";
17        public bool Inline { get; set; } = true;
18    }
19 }
20

```

- Foi alterado o builder de Despesa para incluir a formatação dos novos campos.



```

modelBuilder.Entity<Despesa>().Property(d => d.NomeDocumento)
    .HasMaxLength(100);

modelBuilder.Entity<Despesa>().Property(d => d.HashDocumento);

```

- Foram criados os métodos de verificação de documento duplicado (ConfirmarDocumentoDuplicado), geração de hash SHA-256 (GerarHash) e processamento de documentos (ProcessarDocumento) para salvamento local, renomeamento de documentos pelo hash (SalvarArquivoDocumento), procurar documento por hash no banco (GetByHashDocumento) e obter documento para visualização no navegador (ObterArquivoDocumento) na Service e na Repository de Despesa.

```

/// <summary>
/// Gera um hash SHA-256 de um arquivo de documento.
/// </summary>
/// <param name="arquivo">O arquivo IFormFile para o qual gerar o hash.</param>
/// <returns>Hash SHA-256 em formato hexadecimal.</returns>
1 referência
public static async Task<string> GerarHashAsync(IFormFile arquivo)
{
    if (arquivo == null || arquivo.Length == 0)
    {
        throw new ArgumentException("Arquivo não pode ser nulo ou vazio.");
    }

    using (var sha256 = SHA256.Create())
    using (var stream = arquivo.OpenReadStream())
    {
        var hashBytes = await Task.Run(() => sha256.ComputeHash(stream));
        return Convert.ToHexString(hashBytes);
    }
}

```

```

/// <summary>
/// Confirma se um documento com o mesmo hash já existe no banco de dados.
/// </summary>
/// <param name="hashDocumento">O hash SHA-256 do documento a verificar.</param>
/// <param name="ignoreId">ID de despesa a ignorar na busca (usado em atualizações).</param>
/// <returns>True se o hash já existe, false caso contrário.</returns>
2 referências
public async Task<bool> ConfirmarDocumentoDuplicadoAsync(string hashDocumento, int? ignoreId = null)
{
    if (string.IsNullOrEmpty(hashDocumento))
    {
        throw new ArgumentException("Hash do documento não pode ser nulo ou vazio.");
    }

    return await _despesaRepository.ExistsByHashDocumentoAsync(hashDocumento, ignoreId);
}

```



```

/// <summary>
/// Processa um documento anexado, gerando seu hash e validando duplicatas.
/// Salva o arquivo na pasta "Documentos" com nome baseado no hash.
/// </summary>
/// <param name="despesaDTO">DTO da despesa com o arquivo documento.</param>
/// <param name="ignoreId">ID da despesa a ignorar (usado em atualizações).</param>
/// <exception cref="DespesaValidationException">Lançada se o documento for duplicado.</exception>
2 referências
private async Task ProcessarDocumentoAsync(DespesaDTO despesaDTO, int? ignoreId = null)
{
    if (despesaDTO.Documento == null || despesaDTO.Documento.Length == 0)
    {
        return;
    }

    // Gerar hash do documento automaticamente
    var hashDocumento = await GerarHashAsync(despesaDTO.Documento);

    // Verificar se o hash já existe no banco
    var documentoDuplicado = await ConfirmarDocumentoDuplicadoAsync(hashDocumento, ignoreId);
    if (documentoDuplicado && !despesaDTO.ConfirmarDocumentoDuplicado)
    {
        throw new DespesaValidationException(
            ["Um documento com o mesmo conteúdo já foi cadastrado. Use ConfirmarDocumentoDuplicado=true para sobrescrever."]);
    }

    // Salvar arquivo na pasta Documentos com nome baseado no hash
    await SalvarArquivoDocumentoAsync(despesaDTO.Documento, hashDocumento);

    // Armazenar hash e nome do documento no DTO para serem salvos no banco
    despesaDTO.HashDocumento = hashDocumento;
    despesaDTO.NomeDocumento = despesaDTO.Documento.FileName;
}

```

```

/// <summary>
/// Salva o arquivo de documento na pasta "Documentos" com nome baseado no hash.
/// </summary>
/// <param name="arquivo">O arquivo a ser salvo.</param>
/// <param name="hashDocumento">O hash SHA-256 do documento para uso como nome.</param>
1 referência
private async Task SalvarArquivoDocumentoAsync(IFormFile arquivo, string hashDocumento)
{
    try
    {
        // Obter extensão original do arquivo
        var extensao = Path.GetExtension(arquivo.FileName);

        // Construir caminho da pasta Documentos
        var pastaDocumentos = Path.Combine(_webHostEnvironment.ContentRootPath, DocumentosFolder);

        // Criar pasta se não existir
        if (!Directory.Exists(pastaDocumentos))
        {
            Directory.CreateDirectory(pastaDocumentos);
        }

        // Nome do arquivo: hash + extensão original
        var nomeArquivo = $"{hashDocumento}{extensao}";
        var caminhoCompleto = Path.Combine(pastaDocumentos, nomeArquivo);

        // Salvar arquivo no disco
        using (var stream = new FileStream(caminhoCompleto, FileMode.Create))
        {
            await arquivo.CopyToAsync(stream);
        }
    }
    catch (Exception ex)
    {
        throw new DespesaValidationException(
            ["Erro ao salvar documento: {ex.Message}"]);
    }
}

```

```

3 referências
public async Task<IEnumerable<DespesaDTO>> GetByHashDocumentoAsync(string hashDocumento)
{
    var entities = await _despesaRepository.GetByHashDocumentoAsync(Sanitize(hashDocumento));
    return _mapper.Map<IEnumerable<DespesaDTO>>(entities);
}

```

```

2 referências
public async Task<FileResultDto?> ObterArquivoDocumentoAsync(string hashDocumento)
{
    if (string.IsNullOrEmpty(hashDocumento))
        throw new ArgumentException("Nome do documento não pode ser vazio");

    var despesa = (await GetByHashDocumentoAsync(hashDocumento))
        .FirstOrDefault();

    if (despesa is null)
        return null;

    if (string.IsNullOrEmpty(despesa.HashDocumento))
        throw new InvalidOperationException("Documento não possui hash associado");

    var filePath = Path.Combine(
        _webHostEnvironment.ContentRootPath,
        "Documentos",
        $"{despesa.HashDocumento}.pdf"
    );

    if (!File.Exists(filePath))
        return null;

    var stream = new FileStream(
        filePath,
        FileMode.Open,
        FileAccess.Read,
        FileShare.Read
    );

    return new FileResultDto
    {
        Stream = stream,
        FileName = $"{despesa.HashDocumento}.pdf",
        ContentType = "application/pdf",
        Inline = true
    };
}

```

```

public interface IDespesaService : IGenericService<Despesa, DespesaDTO>
{
    Task ValidarCadastroAsync(DespesaDTO entityDTO, int? id = null);
    Task<IEnumerable<DespesaDTO>> GetByNumeroDocumentoAsync(string numeroDocumento);
    Task<IEnumerable<DespesaDTO>> GetByHashDocumentoAsync(string hashDocumento);
    Task<FileResultDto?> ObterArquivoDocumentoAsync(string nomeDocumento);
    Task<IEnumerable<DespesaDTO>> GetByCodigoAsync(string codigo);
    Task<IEnumerable<DespesaDTO>> GetByUnidadeConsumidoraAsync(int idUnidadeConsumidora);
    Task<IEnumerable<DespesaDTO>> GetByUsuarioAsync(int idUsuario);
    Task<IEnumerable<DespesaDTO>> GetByStatusAsync(Status status);
    Task<DespesaDTO> AlterarStatusAsync(int id, Status novoStatus);
    Task AtualizarDespesasAtrasadasAsync();
    Task<bool> ConfirmarDocumentoDuplicadoAsync(string hashDocumento, int? ignoreId = null);
}

```

```

2 referências
public async Task<IEnumerable<Despesa>> GetByHashDocumentoAsync(string hashDocumento)
{
    var normalized = hashDocumento.Trim().ToUpperInvariant();

    return await _context.Despesas
        .AsNoTracking()
        .Where(d =>
            d.HashDocumento != null &&
            d.HashDocumento.Trim().ToUpper() == normalized
        )
        .ToListAsync();
}

```

2 referências

```
2 referências
public async Task<bool> ExistsByHashDocumentoAsync(string hashDocumento, int? ignoreId = null)
{
    var query = _context.Despesas
        .AsNoTracking()
        .Where(despesa => despesa.HashDocumento == hashDocumento);

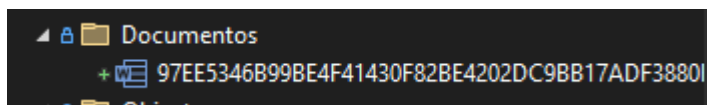
    if (ignoreId.HasValue)
    {
        query = query.Where(despesa => despesa.Id != ignoreId.Value);
    }

    return await query.AnyAsync();
}

using Civitas.WebAPI.Objects.Models;

namespace Civitas.WebAPI.Data.Interfaces
{
    4 referências
    public interface IDespesaRepository : IGenericRepository<Despesa>
    {
        2 referências
        Task<IEnumerable<Despesa>> GetByNumeroDocumentoAsync(string numeroDocumento);
        2 referências
        Task<IEnumerable<Despesa>> GetByHashDocumentoAsync(string hashDocumento);
        2 referências
        Task<IEnumerable<Despesa>> GetByCodigoAsync(string codigo);
        2 referências
        Task<IEnumerable<Despesa>> GetByUnidadeConsumidoraAsync(int idUnidadeConsumidora);
        2 referências
        Task<IEnumerable<Despesa>> GetByUsuarioAsync(int idUsuario);
        2 referências
        Task<IEnumerable<Despesa>> GetByStatusAsync(Status status);
        2 referências
        Task<decimal> SumValorPrevistoByOrcamentoAsync(int idOrcamento, int? ignoreId = null);
        2 referências
        Task<bool> ExistsByHashDocumentoAsync(string hashDocumento, int? ignoreId = null);
    }
}
```

- **Pasta Documentos com Documentos renomeados por hash:**



- Foi alterado o método POST de Despesa para permitir anexo de documento (IFormFile).

```

/// <summary>
/// Cria uma nova despesa no sistema.
/// </summary>
[HttpPost]
public async Task<IActionResult> Post([FromForm] DespesaDTO despesaDTO)
{
    if (despesaDTO is null)
    {
        _response.Code = ResponseEnum.INVALID;
        _response.Data = null;
        _response.Message = "Dados inválidos";

        return BadRequest(_response);
    }

    try
    {
        despesaDTO.Id = 0;

        await _despesaService.Create(despesaDTO);

        _response.Code = ResponseEnum.SUCCESS;
        _response.Data = despesaDTO;
        _response.Message = "Despesa cadastrada com sucesso";

        return Ok(_response);
    }
    catch (DespesaValidationException ex)
    {
        _response.Code = ex.Errors.Any(e => e.Contains("documento com o mesmo conteúdo"))
            ? ResponseEnum.CONFLICT
            : ResponseEnum.INVALID;

        _response.Message = ex.Message;
        _response.Data = ex.Errors;

        return _response.Code == ResponseEnum.CONFLICT
            ? Conflict(_response)
            : BadRequest(_response);
    }
    catch (ArgumentException ex)
    {
        _response.Code = ResponseEnum.INVALID;
        _response.Message = ex.Message;
        _response.Data = null;

        return BadRequest(_response);
    }
    catch (Exception ex)
    {
        _response.Code = ResponseEnum.ERROR;
        _response.Message = "Não foi possível cadastrar a despesa";
        _response.Data = new
        {
            ErrorMessage = ex.Message,
            StackTrace = ex.StackTrace ?? "Sem stack trace disponível"
        };

        return StatusCode(StatusCodes.Status500InternalServerError, _response);
    }
}

```

- Foi criado um método GET de Despesa que permite abrir o documento PDF no próprio navegador para visualização através do parâmetro hash.



```

/// <summary>
/// Abre o documento PDF da despesa no navegador.
/// </summary>
/// <summary>
/// Abre o documento PDF da despesa no navegador pelo nome do documento.
/// </summary>
/// <summary>
/// Abre o documento PDF da despesa no navegador.
/// </summary>
[HttpGet("documento/{hashDocumento}")]
public async Task<ActionResult> AbrirDocumento(string hashDocumento)
{
    if (string.IsNullOrEmpty(hashDocumento))
    {
        _response.Code = ResponseEnum.INVALID;
        _response.Data = null;
        _response.Message = "Hash do documento é obrigatório";

        return BadRequest(_response);
    }

    var fileResult = await _despesaService.ObterArquivoDocumentoAsync(hashDocumento);

    if (fileResult is null)
    {
        _response.Code = ResponseEnum.NOT_FOUND;
        _response.Data = null;
        _response.Message = "Documento não encontrado";

        return NotFound(_response);
    }

    _response.Code = ResponseEnum.SUCCESS;
    _response.Data = new
    {
        fileResult.FileName,
        ContentType = "application/pdf"
    };
    _response.Message = "Documento encontrado com sucesso";

    var encodedFileName = Uri.EscapeDataString(fileResult.FileName);

    Response.Headers.Add(
        "Content-Disposition",
        $"inline; filename*=UTF-8''{encodedFileName}"
    );

    return File(fileResult.Stream, "application/pdf");
}

```

- Foi modificado o Seeder para o registro automático correto de Despesa.

```

var despesaEnergia = new Despesa(
    0,
    "NF-2026-0001",
    "Documento 1",
    "e64d18547b406ae3b00152234dc5327e000d8e7921142b6adcc0ffb8d3be7f3c",
    "1001",
    new DateOnly(2026, 4, 10),
    10450.75m,
    10450.75m,
    1320.50m,
    1320.50m,
    new DateOnly(2026, 4, 25),
    new DateOnly(2026, 4, 24),
    Status.PAGA,
    admin.Id,
    admin,
    unidadeConsumidoraEscola.Id,
    unidadeConsumidoraEscola,
    null
);

var despesaManutencao = new Despesa(
    0,
    "OS-2026-0002",
    "Documento 2",
    "b00e9c91a71899cca2136f9caa6b04321361ed52f801cb6ff4c7d5128cf8be8e",
    "2001",
    new DateOnly(2026, 4, 15),
    9200.00m,
    0m,
    18m,
    0m,
    new DateOnly(2026, 5, 5),
    null,
    Status.A_PAGAR,
    funcionario.Id,
    funcionario,
    unidadeConsumidoraUbs.Id,
    unidadeConsumidoraUbs,
    null
);

await _context.Despesas.AddRangeAsync([despesaEnergia, despesaManutencao], cancellationToken);
await _context.SaveChangesAsync(cancellationToken);

```

## Assinaturas

Documento assinado digitalmente



**JOAO ANTONIO GOMES CESTONARO**  
Data: 16/05/2026 12:18:43-0300  
Verifique em <https://validar.iti.gov.br>

Documento assinado digitalmente



**JOAO PEDRO CHAPICHI**  
Data: 16/05/2026 12:23:11-0300  
Verifique em <https://validar.iti.gov.br>